

A Report About A Cinema Managment System API Using Design Patterns

Alejandro Nuñez, Miguel Sanabria
Software Modeling
Universidad Distrital Francisco José de Caldas

CONTENTS

I	Introduction	2
II	User Stories	2
III	Functional and Nonfunctional Requirements	2
III-A	Functional Requirements	3
III-B	Nonfunctional Requirements	3
IV	Technical Considerations	3
V	UML Diagrams	3
V-A	Sequence Diagrams	3
V-B	Activity Diagrams	5
VI	Architecture	6
VI-A	Deployment Diagram	6
VI-B	ER Database Diagram	6
VI-C	Class Diagram	6
VII	SOLID Principles Analysis	6
VIII	Design Pattern Implementation	7
VIII-A	Creational Patterns	7
VIII-B	Structural Patterns	7
VIII-C	Behavioral Patterns	7
IX	Good Practices and Antipatterns	8
X	Conclusions	8
	References	8

LIST OF FIGURES

1	Sequence Diagram 1	3
2	Sequence Diagram 2	4
3	Sequence Diagram 3	4
4	Sequence Diagram 4	4
5	Sequence Diagram 5	4
6	Sequence Diagram 6	5
7	Sequence Diagram 7	5
8	Activity Diagram 1	5
9	Activity Diagram 2	5
10	Activity Diagram 3	5
11	Activity Diagram 4	5
12	Activity Diagram 5	5
13	Activity Diagram 6	6
14	Activity Diagram 7	6

15	Deployment Diagram	6
16	ER Database Diagram	6
17	Class Diagram	6

A Report About A Cinema Managment System API Using Design Patterns

Abstract—This technical report presents the design and implementation of a RESTful API for a cinema management system using Python, Java, and PostgreSQL. The API supports essential operations such as movie scheduling, ticket booking, user management, and payment processing. The backend is built using Python with FastAPI, handling user authentication, while Java with Spring Boot manages business logic about orders. PostgreSQL serves as the relational database, ensuring data integrity and scalability. The API follows RESTful principles, providing structured endpoints and supporting CRUD operations for movies, screenings, reservations, and payments. Role-based access control (RBAC) is implemented to differentiate privileges among administrators and customers. The adapter pattern is used to integrate different payment gateways, ensuring flexibility in payment processing. The report details the system architecture, database schema, and API documentation using OpenAPI standards. The API is tested using Postman and automated unit tests to ensure reliability and maintainability. This work contributes to the development of scalable and secure cinema management solutions, enabling seamless booking experiences and efficient system administration.

I. INTRODUCTION

Cinema management systems play a crucial role in modern entertainment industries, facilitating efficient operations such as movie scheduling, ticket reservations, user management, and payment processing. With the increasing demand for online booking and automated system functionalities, a robust and scalable backend API is essential to ensure seamless user experiences and efficient administration.

This report presents the development of a RESTful API for a cinema management system, designed using Python, Java, and PostgreSQL. The system is structured to handle critical functionalities, including movie management, seat reservations, and secure payment processing. Python, with the FastAPI framework, is used for handling user authentication, ensuring secure access control. Java, utilizing the Spring Boot framework, is responsible for managing the core business logic about orders, providing reliability and modularity. PostgreSQL serves as the database management system, ensuring data consistency and efficient querying.

The API follows RESTful design principles, supporting CRUD operations for movies, screenings, and bookings while maintaining scalability and performance. The system also integrates an adapter pattern for flexible payment gateway support. This report details the API's architecture, database schema, implementation strategies, and architecture methodologies, providing a comprehensive guide for future improvements and deployment in real-world cinema management scenarios.

II. USER STORIES

This will be a revised version of the problem definition in your proposal.

- **As a user**, I want to browse available movies and showtimes so that I can choose the best option for my schedule and preferences.
- **As a user**, I want to select and reserve my seat when purchasing a ticket so that I can ensure I have the seating arrangement I prefer.
- **As a user**, I want to have a payment gateway so that I can choose a payment method.
- **As a user**, I want to sign up for a membership so that I can receive exclusive discounts and early access to tickets.
- **As a member**, I want to manage my membership and view my rewards so that I can keep track of my benefits and stay updated on perks.
- **As a user**, I want to log in securely and manage my account so that I can update my personal details and view my purchase history easily.
- **As a user**, I want to see a list of movies that are going to be released soon, along with their release dates and length, so that I can plan future visits to the cinema.
- **As a user**, I want to see my cart and add tickets to my cart so that I can pay the order.
- **As an admin**, I want to access a dashboard where I can see a list of all purchases (tickets and food orders) along with user details, so that I can assist customers with any issues.
- **As an admin**, I want to add new films, create showtimes and update showtimes so that I can manage the cinema featured films.

III. FUNCTIONAL AND NONFUNCTIONAL REQUIREMENTS

Requirements are essential elements in the software development process, defining what a system should do and the constraints under which it must operate. They are broadly classified into two categories: functional requirements and non-functional requirements. Functional requirements describe specific behaviors or functions of a system, while non-functional requirements define the system's quality attributes and constraints. Understanding these distinctions is crucial for building reliable and efficient software.

This document outlines the requirements for a cinema management API, which will handle various operations related to movie screenings, ticket bookings, user management, and payment processing.

A. Functional Requirements

The functional requirements of the cinema management API specify the key actions that the system must perform to facilitate efficient cinema operations. These requirements include:

- **User Authentication:** The system must allow users (customers and administrators) to log in using a username and password.
- **Movie Management:** Administrators should be able to add, update, and delete movie listings, including details such as title, duration, genre, and screening times.
- **Showtime Scheduling:** The system must allow admins to define movie schedules, assign theaters, and manage availability.
- **Ticket Booking:** Customers should be able to search for movies, select showtimes, choose seats, and book tickets.
- **Payment Processing:** The system must handle secure online transactions and provide booking confirmation.
- **User Role Management:** Different access levels must be defined for administrators and customers.
- **Report Generation:** The system should generate reports on ticket sales, revenue, and attendance trends.

B. Nonfunctional Requirements

Non-functional requirements (NFRs) define the qualities and constraints that the cinema management API must adhere to, ensuring performance, security, usability, and other critical aspects. Key non-functional requirements include:

- **Performance:** The system should respond to user actions within 2 seconds under normal load.
- **Scalability:** The system must support up to 1,000 concurrent users without degradation in performance.
- **Usability:** The API should be well-documented, allowing developers to integrate it easily with front-end applications.
- **Availability:** The system must have an uptime of at least 99.9
- **Maintainability:** The software should allow for easy updates, bug fixes, and feature expansions with minimal downtime.
- **Data Integrity:** The system should ensure accurate and consistent data storage for movie listings, bookings, and user accounts.

By addressing both functional and non-functional requirements, the cinema management API can provide a robust, secure, and user-friendly experience for administrators and customers alike.

IV. TECHNICAL CONSIDERATIONS

The cinema management API follows a RESTful architecture to ensure simplicity and ease of integration with front-end applications. The backend is developed using Java Spring Boot, which provides a robust framework for handling API requests and managing business logic. Also, the python backend was development using FastAPI in order to make the web services.

The database used is PostgreSQL, offering reliable relational data storage for movies, users, bookings, and transactions. API documentation is provided using Swagger, enabling developers to understand and integrate the API effectively.

Error handling is managed through standardized HTTP status codes and structured error messages, ensuring that failures are properly logged and communicated. Logging is implemented using basic file logging in combination with Spring Boot's built-in logging features to track system activity and debug issues efficiently.

The system is deployed on a local server for development and testing, with the possibility of future cloud deployment for scalability. Continuous deployment is not yet fully automated, but updates are managed manually through Git and local deployments.

V. UML DIAGRAMS

The following UML diagrams illustrate the design and structure of the cinema management API. They provide a visual representation of system components, their interactions, and workflows, ensuring clarity in implementation and future enhancements.

A. Sequence Diagrams

As a user, I want to browse available movies and showtimes so what I can choose the best option for my schedule and preferences

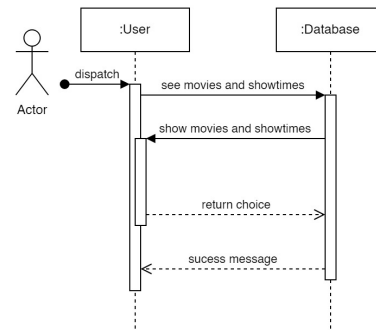


Fig. 1. Sequence Diagram 1

As a user, I want to select and reserve my seat when purchasing a ticket so what I can ensure I have the seating arrangement I prefer.

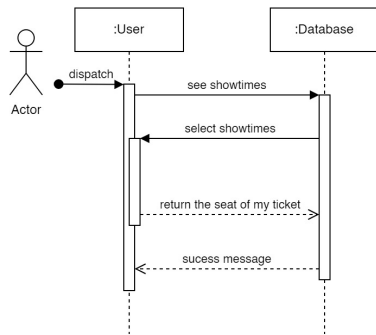


Fig. 2. Sequence Diagram 2

As a member, I want to manage my membership and view my rewards so what I can keep track of my benefits and stay updated on perks.

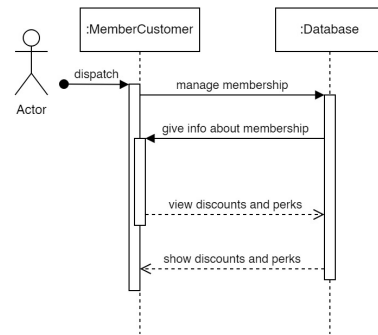


Fig. 4. Sequence Diagram 4

As a user, I want to sign up for a membership so what I can receive exclusive discounts and early access to tickets.

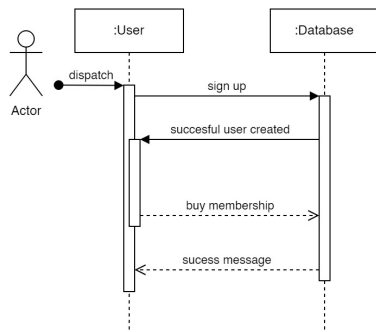


Fig. 3. Sequence Diagram 3

As a user, I want to log in securely and manage my account so what I can update my personal details and view my purchase history easily.

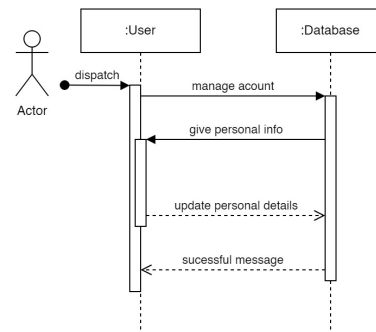


Fig. 5. Sequence Diagram 5

As a user, I want to see a list of movies that are going to be released soon, along with their release dates and length, so what I can plan future visits to the cinema.

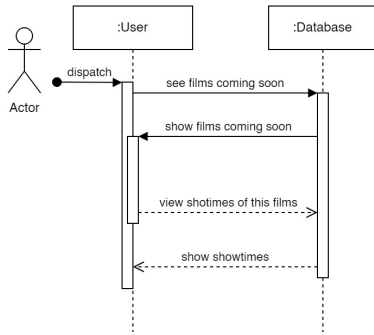


Fig. 6. Sequence Diagram 6

As an admin, I want to access a dashboard where I can see a list of all purchases (tickets and food orders) along with user details, so what I can assist customers with any issues.

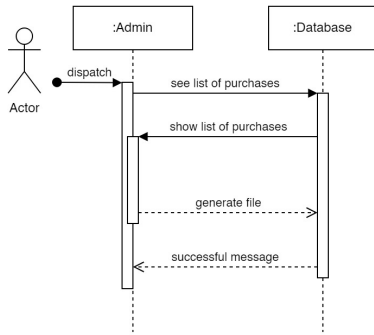


Fig. 7. Sequence Diagram 7

B. Activity Diagrams

As a user, I want to browse available movies and showtimes so what I can choose the best option for my schedule and preferences.

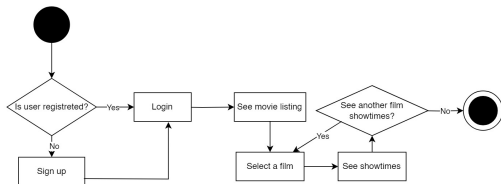


Fig. 8. Activity Diagram 1

As a user, I want to select and reserve my seat when purchasing a ticket so what I can ensure I have the seating arrangement I prefer.

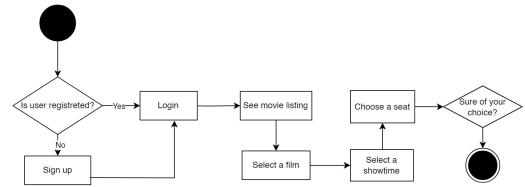


Fig. 9. Activity Diagram 2

As a user, I want to sign up for a membership so what I can receive exclusive discounts and early access to tickets.

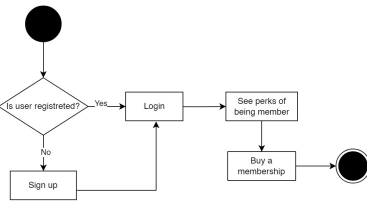


Fig. 10. Activity Diagram 3

As a member, I want to manage my membership and view my rewards so what I can keep track of my benefits and stay updated on perks.

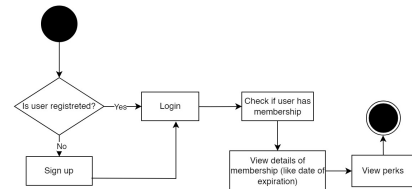


Fig. 11. Activity Diagram 4

As a user, I want to log in securely and manage my account so what I can update my personal details and view my purchase history easily.

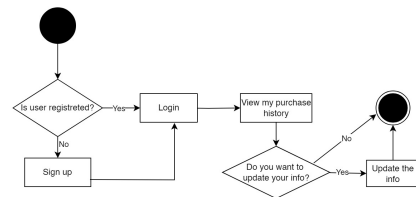


Fig. 12. Activity Diagram 5

As a user, I want to see a list of movies that are going to be released soon, along with their release dates and length, so what I can plan future visits to the cinema.

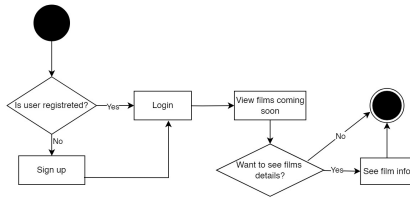


Fig. 13. Activity Diagram 6

As an admin, I want to access a dashboard where I can see a list of all purchases (tickets and food orders) along with user details, so what I can assist customers with any issues.

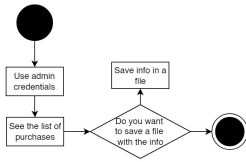


Fig. 14. Activity Diagram 7

VI. ARCHITECTURE

The architecture of the cinema management system defines the structural design and organization of its components. This section presents key diagrams that illustrate the system's deployment, database structure, and object-oriented design. These diagrams help in understanding how different parts of the system interact and how data flows within the application.

A. Deployment Diagram

Deployment Diagram

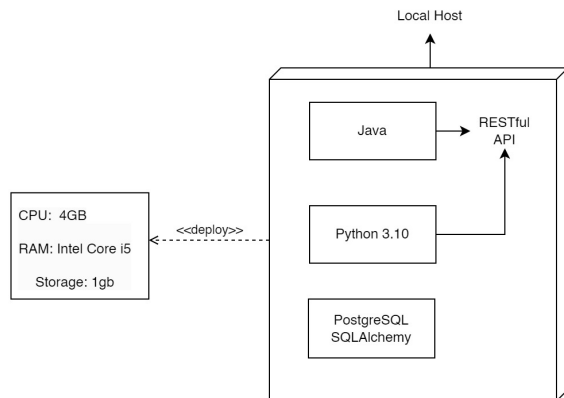


Fig. 15. Deployment Diagram

B. ER Database Diagram

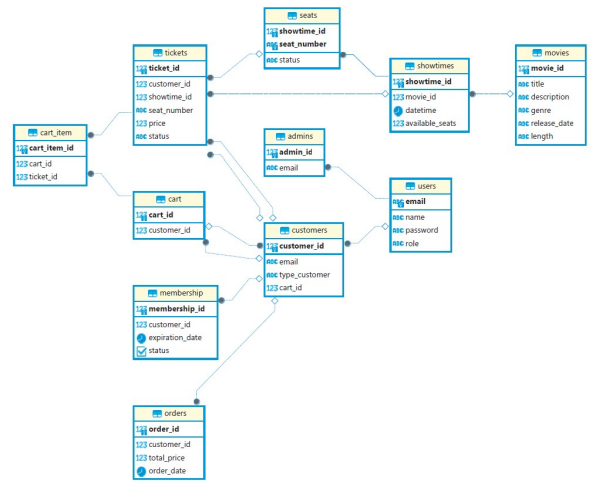


Fig. 16. ER Database Diagram

C. Class Diagram

Class Diagram CineMacondo

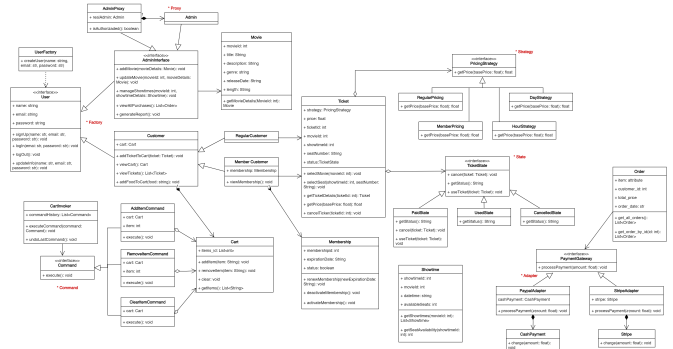


Fig. 17. Class Diagram

VII. SOLID PRINCIPLES ANALYSIS

The SOLID principles ensure that the software architecture remains maintainable, scalable, and easy to extend. The following describes how each principle is applied in the cinema management API:

- **Single Responsibility Principle (SRP):** Each class in the system is designed to have a single responsibility. For example, the `Movie` class handles business logic related to movies, while the `Ticket` class manages ticket reservations. This separation improves maintainability and reduces code coupling.
- **Open/Closed Principle (OCP):** The system is designed to be open for extension but closed for modification. For instance, new payment methods can be added by implementing an interface without modifying existing payment processing logic.

- **Liskov Substitution Principle (LSP):** Subtypes can replace their parent types without affecting the correctness of the program. For example, different types of users (regular customers, member customers and admin) extend a base `User` class while maintaining consistent behavior across the system.
- **Interface Segregation Principle (ISP):** Interfaces are designed to be specific to the needs of their clients. Instead of having a single large interface, the API defines multiple smaller ones, such as `PaymentGateway`, ensuring that classes only implement the methods they actually use.
- **Dependency Inversion Principle (DIP):** High-level modules do not depend on low-level modules but instead rely on abstractions. Dependency injection is used in Spring Boot and FastAPI to manage dependencies, making the system more flexible and easier to test.

VIII. DESIGN PATTERN IMPLEMENTATION

The cinema management API leverages design patterns to improve maintainability, scalability, and flexibility. This section details the applied patterns and their role in the system architecture.

A. Creational Patterns

Creational design patterns define flexible and reusable ways to instantiate objects. The following analyzes the use of these patterns in the cinema management API:

- **Singleton** – Not used. The API does not require a global single instance of any class, as dependency injection in Spring Boot and FastAPI already manages shared resources efficiently.
- **Factory Method** – Used. The API implements a factory method to create users dynamically, distinguishing between customers and administrators. This ensures that new user types can be added without modifying existing code.
- **Abstract Factory** – Not used. The system does not require families of related objects, as user creation is handled via the factory method without the need for multiple related factories.
- **Builder** – Not used. Object construction in this API is straightforward, and there are no complex objects that require step-by-step creation.
- **Prototype** – Not used. The system does not need object cloning since new instances are created directly rather than being copied from existing ones.

B. Structural Patterns

Structural design patterns help define relationships between objects, ensuring that the system remains flexible and maintainable. The following analyzes the use of these patterns in the cinema management API:

- **Proxy** – Used. The API employs a proxy pattern to validate administrator identities before granting access to sensitive operations. This ensures that only authenticated

and authorized users can perform administrative tasks, enhancing security.

- **Adapter** – Used. The API integrates an adapter pattern to connect with different payment gateways. This abstraction allows seamless interaction with multiple external payment services without modifying the core business logic.
- **Facade** – Not used. The API does not require a unified interface for multiple subsystems, as service interactions remain straightforward and manageable.
- **Decorator** – Not used. The system does not rely on dynamic feature extensions at runtime, as functionalities are predefined within their respective classes.
- **Bridge** – Not used. There is no need to separate abstraction from implementation since most services interact directly with their respective implementations.
- **Composite** – Not used. The API does not manage hierarchical structures where individual and composite objects should be treated uniformly.
- **Flyweight** – Not used. The system does not require optimization for memory usage by sharing object states, as performance constraints are not a major concern in this context.

C. Behavioral Patterns

Behavioral design patterns focus on communication between objects and the delegation of responsibilities. The following analyzes the use of these patterns in the cinema management API:

- **Strategy** – Used. The API implements the strategy pattern to determine ticket pricing dynamically. Different pricing strategies are applied based on customer type (regular or client), day of the week (higher cost on weekends, discounts on Tuesdays and Wednesdays), and time of day (higher price after 3 PM). This allows flexible and maintainable pricing adjustments without modifying core logic.
- **State** – Used. The API uses the state pattern to manage ticket status transitions. A ticket can be in different states, such as `paid`, `used`, or `cancelled`, and the behavior of the ticket object changes accordingly. This ensures a structured approach to handling state-dependent logic.
- **Command** – Used. The command pattern is implemented to manage the shopping cart functionality. Users can add, remove, and clear items from the cart using encapsulated command objects. This enables undo/redo capabilities and maintains a history of actions, improving flexibility in user interactions.
- **Observer** – Not used. The API does not require automatic notifications based on state changes, as updates are handled through direct queries rather than event-driven mechanisms.
- **Mediator** – Not used. There is no complex interaction between multiple objects requiring a centralized communication mechanism.

- **Chain of Responsibility** – Not used. The system does not have a chain of handlers processing requests in sequence, as most decisions are made through direct method calls.
- **Template Method** – Not used. The API does not require defining a common algorithm structure with steps that subclasses can override.
- **Memento** – Not used. There is no need for capturing and restoring object states, as changes are handled through database persistence.
- **Interpreter** – Not used. The system does not need to parse and evaluate custom languages or expressions.
- **Visitor** – Not used. The API does not require defining new operations on object structures without modifying their classes.

IX. GOOD PRACTICES AND ANTIPATTERNS

In the cinema management API, several best practices were applied to ensure maintainability and scalability. The Composition Over Inheritance principle was followed by favoring object composition instead of deep inheritance hierarchies, particularly in user role management. The Interface Segregation Principle was applied by designing smaller, focused interfaces, avoiding unnecessary dependencies in components like payment processing. The Principle of Least Knowledge (Law of Demeter) was considered by minimizing direct object dependencies, ensuring modules interacted only with immediate collaborators.

To maintain code quality, hardcoding was avoided by using configuration files and constants, and code duplication was reduced through reusable utility functions. Copy-paste coding was prevented by adhering to DRY (Don't Repeat Yourself), ensuring consistent logic abstraction.

Regarding antipatterns, various code smells were avoided. Long methods and large classes were refactored into smaller, more manageable units. Magic numbers were replaced with named constants for better readability. Dead code was eliminated through regular refactoring, and duplicated code was addressed by extracting shared logic into helper functions. The Feature Envy smell was mitigated by ensuring methods were placed in the appropriate classes rather than accessing excessive external data. Data clumps were handled by encapsulating related data into objects instead of passing numerous parameters.

To avoid refused bequest, inheritance was only used where truly necessary, preventing subclasses from inheriting unused methods. Switch statements were minimized by leveraging polymorphism, especially in ticket pricing strategies. Long parameter lists were reduced through parameter objects, enhancing code readability. Divergent changes and shotgun surgery were prevented by keeping related responsibilities together, ensuring modifications impacted minimal components.

Additionally, inappropriate intimacy and message chains were avoided by encapsulating data access within objects instead of exposing deep object structures. Speculative generality, or overengineering unused features, was controlled by implementing only necessary functionalities, keeping the design practical and adaptable.

X. CONCLUSIONS

In conclusion, the development of the cinema management API successfully applied key software engineering principles to ensure scalability, maintainability, and efficiency. The system follows a RESTful architecture, utilizing Java Spring Boot, FastAPI, and PostgreSQL for backend operations and data management.

The implementation of design patterns improved modularity and flexibility. The Factory Method allowed dynamic user creation, the Proxy pattern ensured secure admin verification, and the Adapter pattern facilitated payment gateway integration. Additionally, Strategy was used for dynamic pricing, State managed ticket statuses, and Command streamlined cart operations.

By adhering to best practices such as composition over inheritance, interface segregation, and the principle of least knowledge, the project maintained clean and efficient code. Furthermore, common code smells and antipatterns, including long methods, duplicated code, magic numbers, and dead code, were identified and avoided to enhance overall code quality.

Overall, the project successfully delivered a structured and scalable API that meets functional requirements while ensuring adaptability for future improvements.

REFERENCES

- [1] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [2] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [3] Eric Freeman, Elisabeth Freeman, Kathy Sierra, Bert Bates. *Head First Design Patterns: A Brain-Friendly Guide*. O'Reilly Media, 2004.
- [4] Martin Fowler, Kent Beck, John Brant, William Opdyke, Don Roberts. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.